

Prima di tutto abilitiamo l'estensione autoreload di Jupyter

```
In [1]: %load_ext autoreload
        %autoreload 2
```

Esercizio: Modello di Beverton-Holt

Il modello di popolazione Beverton-Holt è definito dalla ricorsione:

$$x_{k+1} = \frac{rx_k}{1 + \frac{x_k}{N}}$$

Dove:

- x_k è il valore della popolazione al passo k
- $r \in [1, 2]$ è il tasso di crescita
- N è un valore di popolazione che, se raggiunto, dimezza r

Nel modulo `sol.bh.py`, si definisca la classe:

```
class BevertonHolt:
    def __init__(self, r, N=1):
        ...

    def __call__(self, x, k):
        ...
```

- Il costruttore riceve come ingresso il valore di `r` ed `N`
- Il metodo `__call__` deve calcolare il prossimo valore della popolazione
- Si noti che lo stato in questo caso è uno scalare
- ...Quindi non è necessario usare un array di `numpy`

Nella cella seguente:

- Si utilizzi un ciclo per considerare valori di r tra 1 e 2
- Per ogni valore di r si simuli l'andamento della popolazione usando `sim.simulate`
 - Si utilizzi $x_0 = 0.5$ come valore iniziale della popolazione
 - ...E si considerino 100 passi di simulazione
- Si disegni l'andamento utilizzando `sim.plot_sim`
- Si osservi quali tipi di comportamento può avere il sistema

```
In [6]: from sol import bh
        from base import sim
        import numpy as np

        x0 = 0.5
```

```
n = 100
```

```
for r in np.linspace(1, 2, 6):  
    f = bh.BevertonHolt(r=r) # costruisco la funzione di transizione  
    X = sim.simulate(f, x0, 100) # simulo  
    sim.plot_sim(X, title=f"r = {r:.3f}")
```



